

A Study of BFS, Dijkstra, and A* with Embedding-Based Heuristics on Social Network Graphs

Scott Barcomb

December 2025

Abstract

Shortest path algorithms are a core part of many graph-based applications, but their performance can vary widely depending on graph structure and the effectiveness of available heuristics. Classical algorithms such as Breadth First Search (BFS) and Dijkstra's algorithm guarantee optimal paths but can be very slow on larger graphs. A* search uses a heuristic function, whose effectiveness can greatly increase or decrease performance.

This paper goes over a comparison of BFS, Dijkstra, and A* to verify expected theoretical performance, before investigating the use of an embedding-based heuristic on traversing social network graphs, with the embedding itself being derived from these graphs. Results show an increase in path-finding efficiency by A* using this heuristic when compared to BFS and Dijkstra, but the performance gains are not consistent, and runtime often limited by heuristic overhead. This paper demonstrates the tradeoffs in using embedding-based heuristics for complex networks while highlighting their usefulness.

1 Introduction

The shortest path problem is a fundamental problem in computer science with applications ranging from GPS navigation to network analysis. This paper will analyze and compare classic solutions to this problem such as Breadth First Search (BFS), Dijkstra's algorithm, and A* search, before further exploring the effect that heuristic choices have on large, complex graphs such as those used by social networks.

Classical shortest path algorithms have existed for decades now, and their performance characteristics are well understood as a result. BFS and Dijkstra, for instance, will always find an optimal path, but do so in a naïve fashion, resulting in subpar performance on large graphs [1, 2, 3]. These algorithms prioritize exploring every node until reaching their destination, resulting in a systematic approach that can consume millions of nodes on larger graphs. Since graphs can be weighted or unweighted, however, a key distinction arises between the two. BFS calculates the shortest path in unweighted graphs, while Dijkstra performs the same function in weighted ones

[1, 2]. A* search, on the other hand, is an algorithm that works like Dijkstra’s algorithm, except that at each step it selects a node with the smallest value based on a heuristic [4], allowing the optimal path to be found in a smaller amount of time. Good heuristics can make A* more efficient than other algorithms, while bad heuristics can waste effort and perform worse [5, 6].

Much work has gone into designing new heuristics or coming up with approaches for designing them, given they play a key role in the performance of A*. Research has introduced methods like landmarks (Goldberg and Harrelson) [4] to precompute heuristic information, greatly speeding up searches on large graphs. In large-scale social networks, embedding nodes into geometric spaces has been shown to provide fast distance estimates (Zhao et al.) [3]. Modern research has begun focusing on using machine learning to train heuristic functions from data, such as with the 2022 study by M. Pándy et al. [5].

This paper will compare BFS, Dijkstra, and A* on standard benchmark graphs (grids and SNAP datasets [7]) to collect baseline metrics such as the runtime, memory pressure, and number of nodes expanded (searched through). It will then extend this analysis by experimenting with applying A* on social graph data using a heuristic derived from high-dimensional embeddings, with the goal of evaluating whether such embeddings can guide search efficiently within this domain. This idea was inspired by existing research on distance estimation using graph embeddings [3] and heuristic learning with neural networks [5]. If the embedding captures the complexity of graph structure or geometry well, it could potentially serve as an informative heuristic deserving future research.

2 Methodology

2.1 Experimental Setup

All experiments were conducted on a laptop using C# and the embedding-based heuristic was derived using Python and the NetworkX library, which is a helpful library for the analysis of graph networks [8]. The implementations of Breadth First Search (BFS), Dijkstra’s Algorithm, and A* search were written together in C# to support comparable accuracy, since the algorithms are quite similar.

System Component	Specification for this Project
Operating System	Windows 11
Processor	AMD Ryzen AI 9 365, 10 cores, 2.0 GHz
Memory	24 GB RAM
Language / Libraries	.NET Framework 10.0, Python 3.11, NetworkX 3.5, NumPy

2.2 Datasets

To evaluate the capabilities of each search algorithm, tests were conducted across both synthetic and real-world graphs. The goal of this project was to examine the effectiveness of an embedding-based heuristic on speeding up A* Search on large social graphs, and as such, baseline performance data on these social graphs had to be collected. In addition, comparing the performance of the search algorithms against synthetic graphs helped show how the chosen real-world data may reduce algorithm performance.

The synthetic graphs were programmatically generated to cover basic, standard graph topologies. Grid graphs of size 50x50 and 5000x5000 were used for the study, but the graph dimensions are configurable. These graphs were unweighted to reflect the unweighted nature of the real-world graph.

The real-world dataset was a graph of Facebook users, consisting of roughly 4,000 nodes and 90,000 edges, with nodes representing Facebook users and edges representing that two users were friends on the platform. This graph was selected from the *Stanford Large Network Dataset Collection* (SNAP) [7].

2.3 Algorithm Implementation

BFS was implemented to compute the minimum number of edges between selected nodes. Dijkstra's algorithm was applied in the same fashion, using a priority queue for selecting the next node. A* search was implemented with the same structure as Dijkstra, further incorporating a heuristic function $h(n)$ to aid in node exploration.

Two heuristics were evaluated for A* search:

1. **Manhattan distance** – estimates the cost to the goal by calculating a grid-like straight-line distance between the current node and the goal node.
2. **Embedding-based heuristic** – the research goal of this project, it was computed via node2vec embeddings [9] on the Facebook graph.

2.4 Experimental Procedure

To run the experiment, each algorithm was executed hundreds of times on the test data. A test run consisted of a chosen algorithm, a chosen dataset, and randomly selected start and goal nodes. For each test, the total runtime, the maximum frontier size, and the number of nodes expanded were recorded. Resulting data was then stored in a CSV file for analysis. To ensure results were reproducible, all randomization used a fixed seed.

2.5 Data Collection and Analysis

Data Type	Description
Runtime (milliseconds)	Execution time per run
Nodes Expanded	Total number of nodes visited during search
Max Frontier Size	The total number of nodes or paths under consideration at the widest point in the search

CSV files were used to log all results, and all experiments are configurable via a configuration file. Data was compared across algorithm type, graph type (synthetic vs. real-world), and heuristic choice by using plots. These plots compare the runtime vs. algorithm, the number of nodes expanded vs. algorithm, and the max frontier vs. algorithm. The aim of this analysis was to first show that baseline performance matched theoretical expectations, and to then investigate the strength of the embedding-based heuristic created for this study.

2.6 Availability

All the source code for this project is publicly available in a GitHub repository. The repository includes documentation to reproduce the experiment, including where to source the dataset used and what random seed to use. The link to this repository can be found in this paper's Appendix.

3 Results

The main metric used for comparison was the number of nodes expanded per test run, or query, to represent the amount of work each algorithm went through to find the optimal path. Maximum frontier size and runtime are also present to provide context for memory pressure and practical performance.

3.1 Grid Graph Baselines

For these experiments, two hundred queries were run per algorithm on each graph, with start and end nodes selected randomly, with a minimum distance threshold used to reselect nodes if they were too close together.

As expected, on the 50x50 grid (Figure 1), BFS and Dijkstra had nearly identical performance. They both expanded a large fraction of the grid for most queries, with median values around half the number of nodes in the graph and worst-case runs reaching total graph exploration. This similarity was expected since the graph is unweighted, and without edge weights Dijkstra is the same algorithm as BFS. In contrast, A* using a Manhattan heuristic dramatically reduced the number of nodes expanded. This algorithm's distribution was more tightly clustered around lower values, with a median multiple times smaller than BFS and Dijkstra. This confirms the expected theoretical behavior of a simple heuristic dominating grid graph searches.

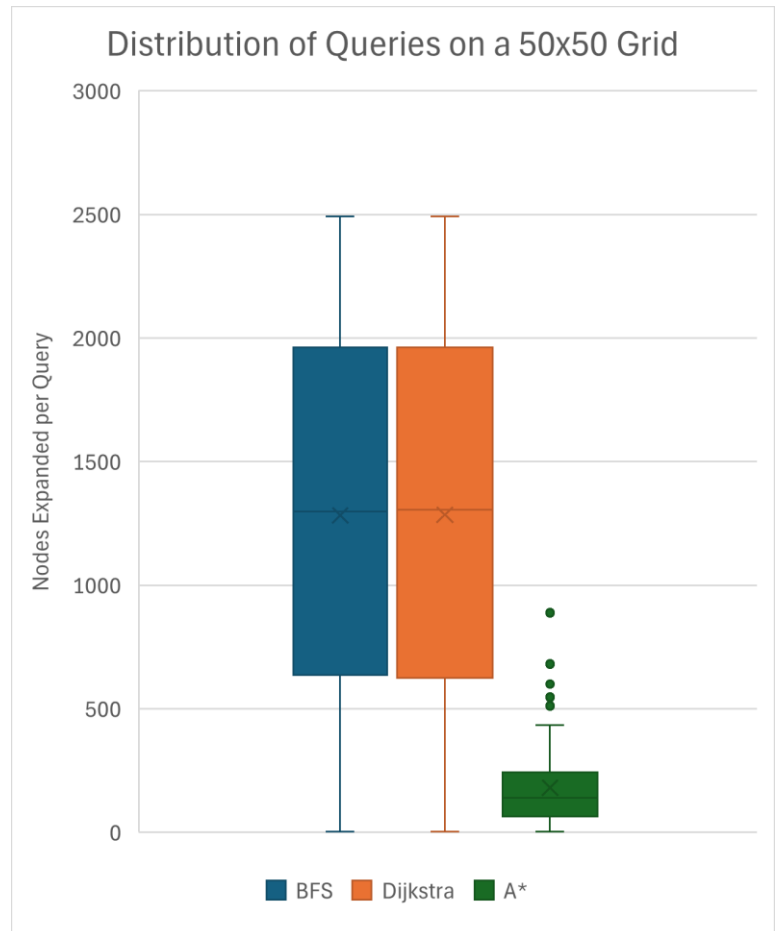


Figure 1

These differences are only increased when scaling to the larger 5000x5000 grid (Figure 2). Again, near identical behavior is shown by BFS and Dijkstra, expanding tens of millions of nodes in the median case. This behavior reflects the quadratic growth of the grid and the resulting harm the naivety of these algorithms can bring with them. A* scaled more favorably: While its absolute number of expanded nodes increased compared to the smaller grid, the total work done continued to remain much lower relative to the uninformed searches. The similarity in results between the two grid sizes produced a key insight regarding heuristic-driven search – the search effort does not grow with the size of the graph, but rather with the difficulty of the path (i.e., the graph topology).

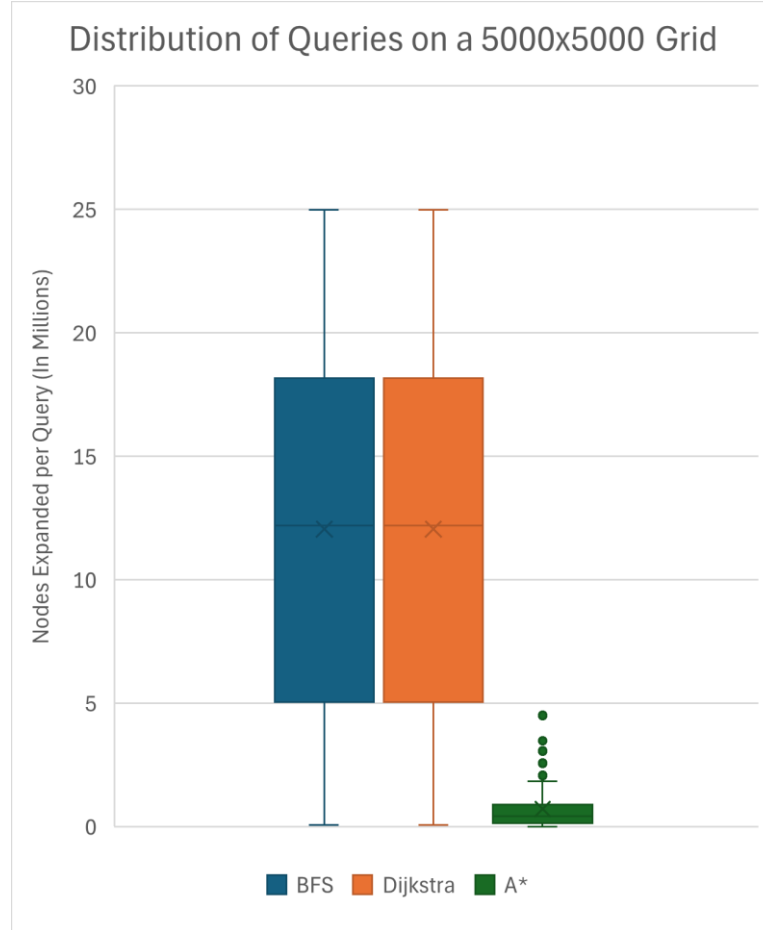


Figure 2

3.2 Social Network Graph: Nodes Expanded

We next evaluated the performance on a real-world Facebook social network graph from the SNAP dataset, which consists of social circles, or “friends lists” from Facebook with approximately 4,000 nodes and 90,000 edges. Unlike grid graphs, the topology of this network lacks a clear geometric structure, making heuristic design for A* search much more challenging.

On this graph (Figure 3), the BFS and Dijkstra algorithms again produced nearly indistinguishable distributions of nodes expanded, due to the unweighted nature of the network. Both algorithms expanded a substantial portion of the graph for most queries, with runs that approached full graph traversal. This can be seen as a reflection of the real-world nature of social networks – shortest paths are typically short, hovering around four to five nodes, but discovering them without guidance requires wide exploration.

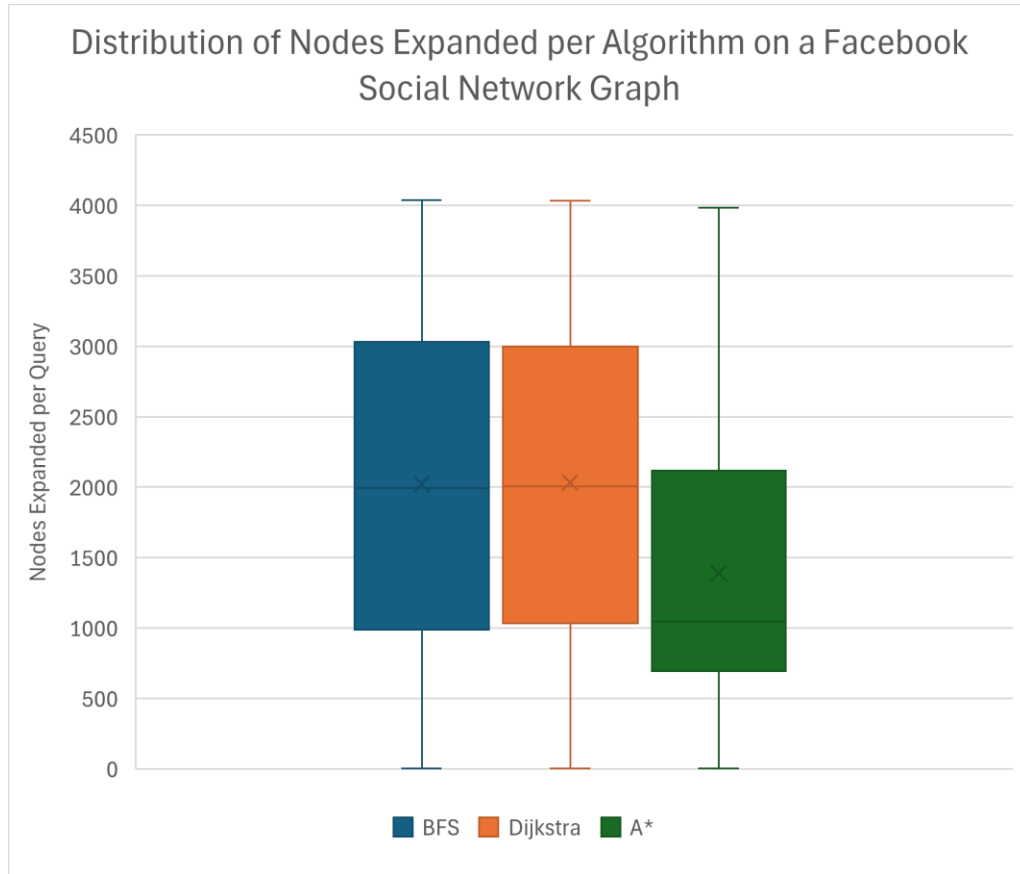


Figure 3

To combat this difficulty, A* search was fed a heuristic created with a node2vec embedding of the network, which showed moderate success in reducing the number of node expansions relative to BFS and Dijkstra. The median number of expanded nodes was much lower, and the distribution shifted downwards in general. However, the variance still remained large, with the worst-case performance still expanding the entire graph. These results suggest that the embedding heuristic captured some useful information about the global structure of the graph but failed to provide universally strong estimates across all node pairs. Nevertheless, the reduction in median expansions gives credence to embeddings being able to function as informative heuristics in social network graphs.

3.3 Social Network Graphs: Frontier Size and Runtime

Maximum frontier size represents the number of nodes or paths that were considered for expansion at the widest step, and as such provides insight into peak memory pressure during search. Testing for this metric on the Facebook graph (Figure 4), BFS showed the largest variation in frontier sizes, while Dijkstra and A* showed some reduction. A* did not consistently minimize the frontier size.

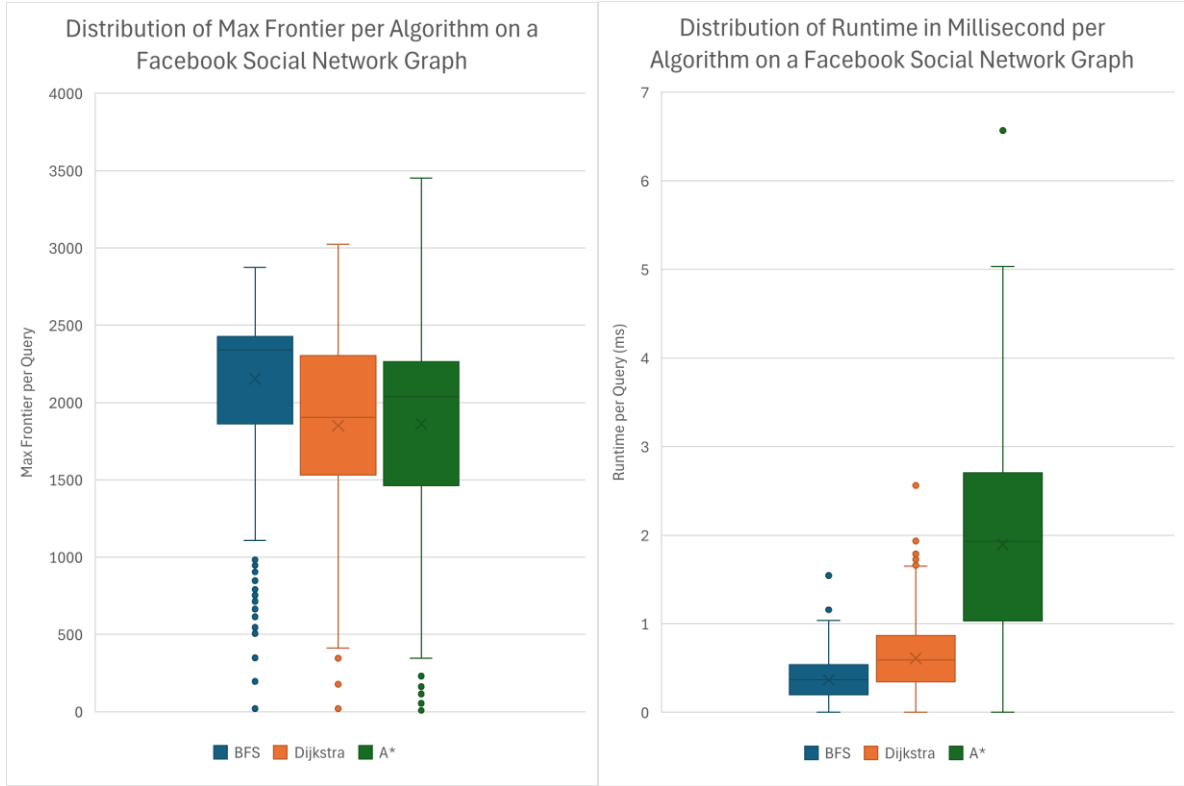


Figure 4

For runtime, the measurements largely correlated with algorithmic overhead. Despite expanding fewer nodes overall, A* was consistently the slowest performing algorithm among the three. BFS utilizes a simple queue-based implementation, which allowed it to achieve the lowest median runtime, while Dijkstra suffered moderate overhead from its priority queue-based implementation, which was unneeded due to the unweighted nature of the graph.

Overall, the results demonstrate the ability for A* to decrease search effort on both synthetic and real-world graphs when using an appropriate heuristic, with a core tradeoff: Heuristic-guided search reduces exploration but introduces computational overhead on large-scale graphs.

4 Related Work

To begin with, it is important to acknowledge that shortest path computation is a well-studied problem in graph theory. Breadth First Search (BFS) and Dijkstra's algorithm are time-tested approaches that guarantee optimal paths in unweighted and weighted graphs, respectively [10, 2]. Both methods are, however, completely uninformed; the expansion of nodes solely based on distance from the source can lead to excessive exploration on large, complex graphs. This is the core motivator for heuristic research and development.

The A* search algorithm was first proposed by Hart, Nilsson, and Raphael [11] in 1968, introducing the heuristic function to Dijkstra's algorithm. When the heuristic is admissible (never

overestimates the remaining cost), A* will guarantee an optimal path in the same fashion as BFS and Dijkstra, but more efficiently. Work on A* first focused on simple domains, such as grid graphs, because geometric heuristics like the Euclidean distance function provide strong results there [12]. These heuristics are computationally inexpensive and serve as useful baselines in pathfinding applications, like robotics and navigation systems.

The performance of geometric heuristics starts to break down in domains that lack explicit geometric structure, such as social networks, which has led to the development of increasingly sophisticated techniques for speeding up shortest path search on such graphs. One notable line of work focuses on preprocessing-based heuristics; Goldberg and Harrelson [13] introduced the ALT framework, which makes use of the distances between preselected landmark nodes in a graph and the triangle inequality to derive admissible heuristics. Further work using this technique demonstrated speedups on road networks [14]. These approaches, however, rely on domain-specific assumptions and preprocessing, making them unsuitable for more dynamic graphs.

In social networks and other small-world graphs, shortest path searching becomes even more challenging. Zhao et al. [3] demonstrated this problem, citing the lack of an underlying geometry in these graphs making path discovery more inefficient, even though the shortest paths are often only a few nodes long. As a result, classical algorithms like BFS often require the exploration of a large portion of the graph before reaching the target.

Graph embeddings are one technique that aims to solve this challenge, with methods like DeepWalk [15], LINE [16], and node2vec [9] working to embed nodes into low-dimensional vector spaces. This technique has been widely used for tasks such as node classification, link prediction, and community detection. Previous work has also investigated embeddings that work to preserve graph distances and proximity relationships [18], suggesting that distances in embedding space could serve as an approximation for the distances used in graph theory.

More recent work has investigated the use of learned or data-driven heuristics for graph search. Pándy et al. [5] show that heuristics that are learned from data can outperform handmade heuristics in some search domains, while other related work uses neural networks to guide searches [17]. While these methods show strong performance, they often require supervised training and significant computational resources, which limits their practicality for general-purpose graph searching.

In contrast, embedding-based heuristics stand as a middle ground between handmade and fully learned approaches. There is no requirement for labeled shortest-path data (as is needed in supervised learning) and they still capture structural information about the graph they are trained on. While previous work has shown some potential for their use with distance estimation [3], little empirical work has studied their effectiveness as heuristics for classical search algorithms such as A*. This gap motivated the current study, which investigates whether node embeddings

that are derived from social networks can meaningfully guide A* search and reduce exploration compared to uninformed baselines such as BFS and Dijkstra.

5 Conclusion

This paper focused on understanding how heuristic choice affects search efficiency when compared to classical shortest path algorithms, before it analyzed how embedding-based heuristics could be useful for path finding on less structured graphs such as those that are representative of social networks. In particular, Breadth First Search (BFS), Dijkstra’s algorithm, and A* search were compared, and a SNAP provided social network graph of Facebook users was used.

Experiments on consistently structured grid graphs were performed to serve as a baseline validation of each algorithm’s performance. In this domain, BFS and Dijkstra demonstrated how uninformed searches result in large portions of the graph being explored before an optimal path is found, whereas A* with a Manhattan-distance heuristic produced the same result in a much more efficient manner. These results matched theoretical expectations that an admissible heuristic can focus exploration much more accurately towards the goal, and these experiments served as a control for demonstrating the expected effect of proper heuristic guidance.

With these results in mind, experiments were then conducted on the Facebook social network graph to explore whether an embedding-based heuristic could produce comparable results. As expected, the uninformed search algorithms, namely BFS and Dijkstra, showed similar behavior to the control experiments, searching through huge portions of the graph before finding the optimal path. A*, with its new heuristic derived from an embedding of the graph itself, was able to often reduce this search scope, but not as consistently as it had in the control experiments, with some outlying runs performing just as poorly as the uninformed searches. These results indicate that even embeddings trained without supervision can function as informative heuristics, but further optimizations can still be made.

As a whole, the results demonstrate that heuristic-guided search is still effective for improving shortest path computation, but its level of success depends on how well the heuristic aligns to the task at hand, making heuristic engineering an enduring challenge. Additionally, the experiments revealed practical tradeoffs in uninformed versus informed approaches to graph traversal, the key point being that a reduction in node expansion does not directly translate to a reduction in runtime. Rather, the runtime of the algorithm is tied up in computational overhead intrinsic to each algorithm’s implementation, and their performance will vary across systems.

Future work could explore more in-depth analysis of embedding-based heuristics, including a more supervised learning dominant approach. Another direction to take would be to investigate hybrid approaches, which would involve creating heuristics optimized for both runtime and efficient node expansion.

Bibliography

- [1] L. Lee, J. Siek, and A. Lumsdaine, "The Generic Graph Component Library," *ACM SIGPLAN Notices*, August 1999, DOI: 10.1145/320385.320428
- [2] S. Sryheni and K. Gilmore. "Difference Between BFS and Dijkstra's Algorithms." Baeldung. <https://www.baeldung.com/cs/graph-algorithms-bfs-dijkstra> (accessed Sep. 31, 2025).
- [3] X. Zhao, A. Sala, H. Zheng, and B. Zhao, "Efficient Shortest Paths on Massive Social Graphs," PhD Thesis, Dept. Computer Science, U.C. Santa Barbara, Santa Barbara, CA, USA, 2011.
- [4] A. Goldberg and C. Harrelson, "Computing the Shortest Path: A^* Search Meets Graph Theory," Microsoft Research, Redmond, WA, USA, Rep. MSR-TR-2004-24, 2003.
- [5] M. Pándy et al., "Learning Graph Search Heuristics," in *The First Learning on Graphs Conference*, B. Rieck and R. Pascanu, Vol. 198, Virtual, Dec.2022, pp. 1-13.
- [6] N. Mitic. "Comparing BFS, DFS, Dijkstra, and A^* algorithms on a practical maze solver example." NemanjaMitic. <https://nemanjamitic.com/blog/2025-07-31-maze-solver> (accessed Sep. 31, 2025).
- [7] J. Leskovec and A. Krevl, "Stanford Large Network Dataset Collection." (Jun. 2014). Distributed by Stanford University. <https://snap.stanford.edu/data/> (accessed Sep. 31, 2025).
- [8] "NetworkX", 2025. Available: <https://networkx.org/documentation/stable/index.html>
- [9] J. Leskovec and A. Grover, "node2vec: Scalable Feature Learning for Networks," *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, July 2016, DOI: 10.48550/arXiv.1607.00653
- [10] E.W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math*, vol. 1, pp. 269-271, 1959, DOI: 10.1007/BF01386390
- [11] P.E. Hart, N.J. Nilsson, B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100-107, July 1968, DOI: 10.1109/TSSC.1968.300136
- [12] J. Pearl, *Heuristics: intelligent search strategies for computer problem solving*. Boston, MA, United States: Addison-Wesley Longman Publishing Co., Inc., 1984.
- [13] A.V. Goldberg and C. Harrelson, "Computing the shortest path: A search meets graph theory," *Proceedings of the sixteenth annual ACM-SIAM symposium on Discrete algorithms*, pp. 156-165, Jan. 2005, DOI: 10.5555/1070432.1070455

- [14] H. Bast, S. Funke, P. Sanders, D. Schultes, “Fast Routing in Road Networks with Transit Nodes,” *Science*, vol. 316, no. 5824, p. 566, Apr. 2007, DOI: 10.1126/science.1137521
- [15] B. Perozzi, R. Al-Rfou, S. Skiena, “DeepWalk: online learning of social representations,” *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 701-710, Aug. 2014, DOI: 10.1145/2623330.2623732
- [16] J. Tang et al., “LINE: Large-scale Information Network Embedding,” *Proceedings of the 24th International Conference on World Wide Web*, pp. 1067-1077, May 2015, DOI: 10.1145/2736277.2741093
- [18] A. Tamar et al., “Value iteration networks,” *Proceedings of the 30th International Conference on Neural Information Processing Systems*, pp. 2154-2162, Dec. 2016, DOI: 10.5555/3157096.3157337

6 Appendix

6.1 Declaration of Authorship

I attest with my signature that I have completed this paper independently and without any assistance from third parties and that the information concerning the sources used in this paper is true and complete in every respect. All sources that have been quoted or paraphrased have been referenced accordingly.

Additionally, I affirm that any text passages written with the help of AI-supported technology are marked as such, including a reference to the AI-supported program used.

This paper may be checked for plagiarism and use of AI-supported technology using appropriate software. I understand that unethical conduct may lead to a grade of 1 or “fail” or to expulsion from the course of studies.

I have taken note of the fact that in the event of a justified suspicion of the unauthorized or undisclosed use of AI in written performance assessments, I am upon request obligated to cooperate in confirming or ruling out the suspicion, for example by attending an interview.

2.1 Record of AI Tools Used

Tools	Tasks	Section
ChatGPT 5.2, OpenAI	Assistance for literary definitions, Python syntax support	Methods, <i>generate_node2vec.py</i>

2.2 Final Paper Word Count Table

Section	MS Word Wordcount
Introduction	449
Methods	623
Results	768
Related Work	612
Conclusion	420
Subtotal: 2,700 min/3,300 max	2,872
Abstract	150
IEEE Bibliography	416
Declaration of authorship	160
Record of AI Tools Used	21
Final paper wordcount table	48
Total	3693

2.3 GitHub Repository with All Project Code

<https://github.com/scottbarcomb/ShortestPaths>